

Université Hassan II de
Casablanca
Faculté des Sciences & Techniques
Mohammedia

Licence Informatique, Réseaux & Multimedia

Traitement des Données Multimodales

Prétraitement des Données

Guide Complet du Prétraitement des Données Textuelles, Visuelles et Audio

Réalisé par :
Prof. Abdellah ADIB

June 4, 2026

Contents

1	Introduction	3
1.1	Contexte et Objectifs	3
1.2	Les Trois Modalites de Prétraitement	3
1.3	Le Pipeline de Prétraitement Multimodal	3
2	Prétraitement Textuel	5
2.1	Principe Fondamental	5
2.1.1	Architecture du Pipeline TAL	5
2.2	Installation et Configuration	5
2.3	Nettoyage des Données	6
2.3.1	Suppression des caractères spéciaux et normalisation de la casse	6
2.3.2	Gestion des valeurs manquantes et doublons	8
2.4	Tokenisation	8
2.5	Filtrage des Stopwords	10
2.6	Normalisation Morphologique	11
2.6.1	Racinisation (Stemming)	11
2.6.2	Lemmatisation	11
2.7	Encodage et Embeddings	13
2.8	Analyse Exploratoire (EDA) du Texte	13
2.9	Bonnes Pratiques	14
3	Prétraitement Visuel	15
3.1	Principe Fondamental	15
3.1.1	Architecture du Pipeline Vision	15
3.2	Installation	15
3.3	Redimensionnement et Transformations Géométriques	16
3.4	Conversion Calorimétrique et Normalisation	17
3.5	Filtrage et Réduction de Bruit	18
3.6	Amélioration du Contraste	19
3.7	Détection de Contours	20
3.8	Bonnes Pratiques	21
4	Prétraitement Audio	22
4.1	Principe Fondamental	22
4.1.1	Architecture du Pipeline Audio	22
4.2	Installation	22
4.3	Rééchantillonnage	23
4.4	Pre-emphase	24
4.5	Framing et Windowing	25
4.6	Détection d'activité Vocale (VAD)	27
4.7	Normalisation du Signal	27

4.8	Bonnes Pratiques	29
5	Synthese et Bonnes Pratiques	30
5.1	Tableau Comparatif des Techniques de Prétraitement	30
5.2	Erreurs Frequentes et Solutions	30
5.3	Checklist de Reproductibilite	31
5.4	Ressources Complémentaires	31

Licence IRM

Chapitre 1 Introduction

1.1 Contexte et Objectifs

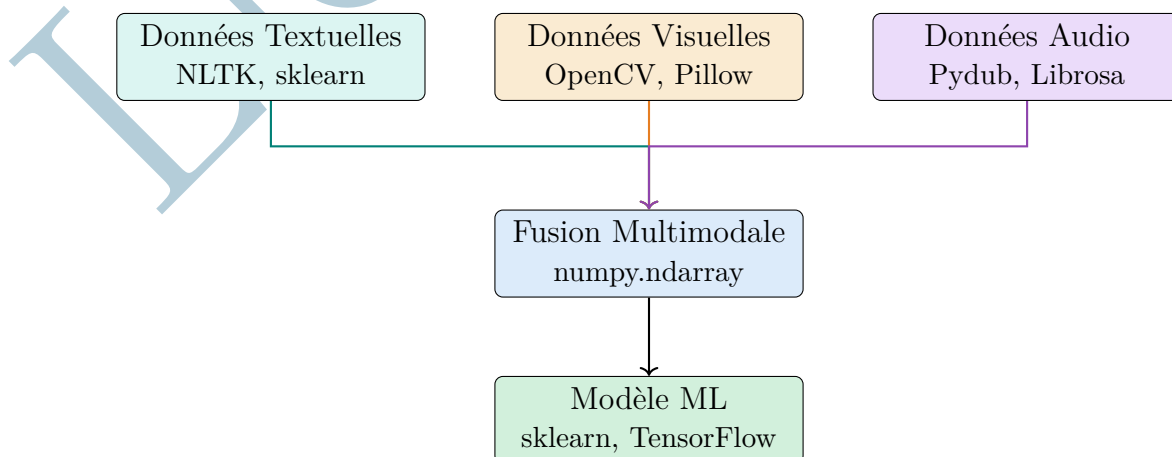
Le prétraitement des données est l'étape fondamentale qui transforme des données brutes — textes, images, signaux audio — en représentations numériques normalisées exploitables par les algorithmes d'apprentissage automatique. Quelle que soit la modalité traitée, les données brutes contiennent invariablement du bruit, des artefacts et des hétérogénéités qui dégradent les performances des modèles.

Ce rapport présente un guide pratique du prétraitement multimodal structure en trois chapitres correspondant aux trois modalités principales : le texte, l'image et l'audio. Chaque chapitre expose les concepts fondamentaux, les bibliothèques Python de référence et les pipelines complets avec exemples de code.

1.2 Les Trois Modalités de Prétraitement

1. **Prétraitement Textuel** : nettoyage, tokenisation, filtrage des stopwords, normalisation morphologique (stemming, lemmatisation) et encodage numérique (TF-IDF, embeddings). Bibliothèques : NLTK, scikit-learn, Gensim, Transformers.
2. **Prétraitement Visuel** : redimensionnement, conversion d'espace colorimétrique, réduction de bruit, amélioration du contraste (CLAHE) et détection de contours. Bibliothèques : OpenCV, Pillow, NumPy.
3. **Prétraitement Audio** : rééchantillonnage, pré-emphase, framing, windowing, détection d'activité vocale (VAD) et normalisation. Bibliothèques : Pydub, Librosa, SciPy.

1.3 Le Pipeline de Prétraitement Multimodal



Conseil

Choisissez la chaîne de prétraitement en fonction de la **nature** de la donnée :

- Donnee = texte, sous-titres, transcriptions → Pipeline TAL (Chapitre 1)
- Donnee = image, vidéo, frame → Pipeline Vision (Chapitre 2)
- Donnee = son, parole, musique → Pipeline Audio (Chapitre 3)

Licence IRM

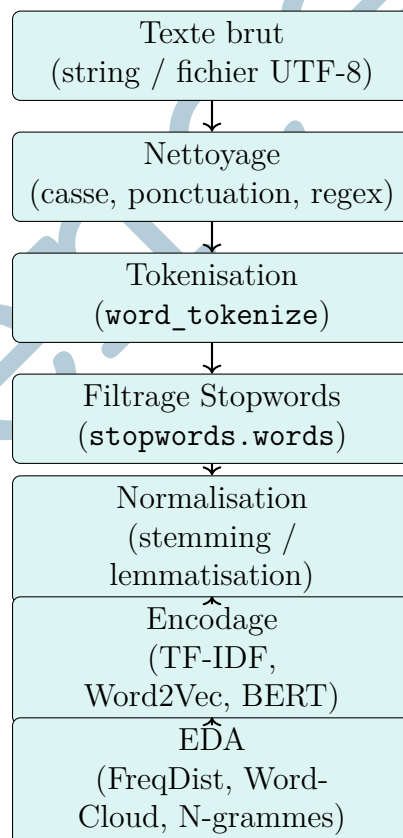
Chapitre 2 Prétraitement Textuel

2.1 Principe Fondamental

Le prétraitement textuel transforme un corpus de texte brut en représentations numériques exploitables par un modèle d'apprentissage. Dans les projets multimodaux, il s'applique aux sous-titres, transcriptions (issues de Whisper ou d'un STT), annotations et commentaires associés aux données image et audio.

NLTK (Natural Language Toolkit) est la bibliothèque Python historique de référence pour le TAL : elle fournit des outils de tokenisation, filtrage des stopwords, racinisation et lemmatisation. Elle est complétée par **scikit-learn** pour la vectorisation (TF-IDF), **Gensim** pour les embeddings denses (Word2Vec, GloVe) et **Transformers** (Hugging-Face) pour les modèles contextuels (BERT, CamemBERT).

2.1.1 Architecture du Pipeline TAL



2.2 Installation et Configuration

Listing 2.1: Installation des bibliothèques TAL

```
pip install nltk scikit-learn gensim transformers
```

Listing 2.2: Téléchargement des ressources NLTK

```
import nltk

# Ressources indispensables (à exécuter une seule fois par
# machine)
nltk.download('punkt')          # tokeniseur de phrases et de mots
nltk.download('stopwords')      # listes de mots vides (40+
# langues)
nltk.download('wordnet')        # base lexicale pour la
# lemmatisation
nltk.download('averaged_perceptron_tagger_eng') # POS tagging
# anglais
```

Attention

Sans les ressources NLTK téléchargées, tout appel à `word_tokenize`, `stopwords.words` ou `WordNetLemmatizer` lèvera une erreur `LookupError`. Cette étape est obligatoire avant toute utilisation.

2.3 Nettoyage des Données

2.3.1 Suppression des caractères spéciaux et normalisation de la casse

Définition

Le **nettoyage textuel** désigne l'ensemble des opérations qui éliminent le bruit d'un texte brut : mise en minuscules, suppression de la ponctuation, retrait des chiffres, des balises HTML et des caractères spéciaux. C'est la **première étape obligatoire** de tout pipeline TAL.

Utilité

Un texte non nettoyé génère des tokens parasites (“!!”, “123”, “
”) qui augmentent la dimensionnalité du vocabulaire sans apporter de valeur sémantique. Le nettoyage réduit le vocabulaire de 20 à 40 % et amélioré la qualité des représentations.

Fonction	Fonctionnalité	Instruction
<code>str.lower()</code>	Convertir en minuscules	<code>texte.lower()</code>
<code>str.strip()</code>	Supprimer espaces bordure	<code>texte.strip()</code>
<code>re.sub()</code>	Supprimer caractères spéciaux	<code>re.sub(r'[^a-z0-9\s]', '', t)</code>

Fonction	Fonctionnalite	Instruction
<code>re.sub()</code>	Supprimer les chiffres	<code>re.sub(r'\d+', "", texte)</code>
<code>re.sub()</code>	Supprimer balises HTML	<code>re.sub(r'<.*?>', "", texte)</code>
<code>str.replace()</code>	Remplacer une sous-chaîne	<code>texte.replace('\n', ' ')</code>
<code>isalpha()</code>	Tester si alphabetique	<code>token.isalpha()</code>
<code>isalnum()</code>	Tester si alphanumerique	<code>token.isalnum()</code>

Listing 2.3: Pipeline de nettoyage textuel complet

```

import re

def nettoyer_texte(texte: str) -> str:
    """
    Applique un pipeline de nettoyage standard sur un texte brut.

    Etapes :
    1. Mise en minuscules
    2. Suppression des balises HTML
    3. Suppression des caracteres speciaux
    4. Suppression des chiffres isoles
    5. Normalisation des espaces multiples

    Returns:
    Texte nettoye pret pour la tokenisation
    """
    # 1. Minuscules
    texte = texte.lower()

    # 2. Suppression balises HTML
    texte = re.sub(r'<.*?>', ' ', texte)

    # 3. Remplacement des apostrophes typographiques
    texte = texte.replace('\u2019', '"').replace('\u2018', '"')

    # 4. Suppression des caracteres speciaux (conservation lettres, chiffres, espaces)
    texte = re.sub(r'[^w\s]', ' ', texte)

    # 5. Suppression des chiffres isoles
    texte = re.sub(r'\b\d+\b', '', texte)

    # 6. Normalisation des espaces multiples
    texte = re.sub(r'\s+', ' ', texte).strip()

    return texte

```



```
# Exemple
brut = "<p>Le modele NLP-2024 analyse <b>3 corpus</b> en
      francais!</p>"
propre = nettoyer_texte(brut)
print(propre)
# "le modele nlp analyse corpus en francais"
```

2.3.2 Gestion des valeurs manquantes et doublons

Listing 2.4: Gestion des valeurs manquantes et doublons textuels

```
import pandas as pd

def nettoyer_corpus_dataframe(df: pd.DataFrame,
                             col_texte: str = 'texte') ->
    pd.DataFrame:

    """
    Nettoie un DataFrame contenant une colonne de textes.

    Args:
        df          : DataFrame avec une colonne de textes
        col_texte   : Nom de la colonne textuelle

    Returns:
        DataFrame nettoye sans NaN ni doublons
    """
    n_initial = len(df)

    # 1. Suppression des lignes sans texte (NaN, None, chaines
    #      vides)
    df = df.dropna(subset=[col_texte])
    df = df[df[col_texte].str.strip() != '']

    # 2. Suppression des doublons textuels exacts
    df = df.drop_duplicates(subset=[col_texte])

    # 3. Suppression des textes trop courts (< 10 caracteres)
    df = df[df[col_texte].str.len() >= 10]

    print(f"Lignes initiales      : {n_initial}")
    print(f"Lignes conservees      : {len(df)}")
    print(f"Lignes supprimees          : {n_initial - len(df)}")

    return df.reset_index(drop=True)
```

2.4 Tokenisation

Définition

La **tokenisation** découpe un texte brut en unités élémentaires appelées **tokens**. Un token peut être un mot, un nombre, un signe de ponctuation ou une expression entière. C'est la **première étape linguistique** après le nettoyage.

Exemple : "Le traitement multimodal." → ["Le", "traitement", "multimodal", "."]

Utilité

Sans tokenisation, un texte est une chaîne inutilisable par un algorithme ML. Elle permet d'isoler chaque unité lexicale pour le comptage de fréquences, l'étiquetage grammatical et la vectorisation (TF-IDF, sac de mots).

Fonction	Fonctionnalité	Instruction
<code>word_tokenize()</code>	Tokeniser en mots	<code>word_tokenize(texte, language='french')</code>
<code>sent_tokenize()</code>	Tokeniser en phrases	<code>sent_tokenize(texte, language='french')</code>
<code>RegexTokenizer()</code>	Tokeniser par regex	<code>RegexTokenizer(r'\w+').tokenize(texte)</code>
<code>MWETokenizer()</code>	Expressions multi-mots	<code>MWETokenizer(['New', 'York'])</code>
<code>TweetTokenizer()</code>	Tokeniser des tweets	<code>TweetTokenizer().tokenize(texte)</code>

Listing 2.5: Tokenisation et filtrage

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize

texte = ("Le pretraitement multimodal est essentiel. "
        "Il combine texte, images et audio.")

# Tokenisation en phrases
phrases = sent_tokenize(texte, language='french')
print(f"Phrases : {phrases}")

# Tokenisation en mots
tokens = word_tokenize(texte, language='french')
print(f"Tokens ({len(tokens)}) : {tokens}")

# Filtrage : conservation des tokens alphabetiques uniquement
tokens_nets = [t.lower() for t in tokens if t.isalpha()]
print(f"Nets ({len(tokens_nets)}) : {tokens_nets}")
# ['le', 'pretraitement', 'multimodal', 'est', 'essentiel',
...]
```

2.5 Filtrage des Stopwords

Définition

Les **stopwords** (mots vides) sont des mots très fréquents mais peu porteurs de sens sémantique : articles, prépositions, pronoms, conjonctions (*le, la, de, et, est, un, dans, que...*). Leur suppression réduit le vocabulaire sans dégrader la qualité des représentations.

Utilité

Supprimer les stopwords réduit le vocabulaire de 30 à 50 % tout en concentrant les représentations sur les termes informatifs. Un modèle TF-IDF sans filtrage est domine par des mots comme « le » ou « de » qui ne distinguent pas les documents.

Fonction	Fonctionnalité	Instruction
<code>stopwords.words('fr')</code>	Stopwords français (157 mots)	<code>set(stopwords.words('french'))</code>
<code>stopwords.words('en')</code>	Stopwords anglais (179 mots)	<code>set(stopwords.words('english'))</code>
<code>stopwords.fileids()</code>	Langues disponibles (40+)	<code>stopwords.fileids()</code>
<code>sw.update()</code>	Ajouter des mots personnalisés	<code>sw.update(['multimodal', 'donnée'])</code>

Listing 2.6: Filtrage des stopwords et extension du lexique

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

texte = "Le modele analyse les donnees multimodales de la
conference."
tokens = word_tokenize(texte, language='french')

# Chargement des stopwords francais
sw_fr = set(stopwords.words('french'))

# Extension : ajout de mots domaine-specifiques
sw_fr.update(['donnee', 'modele', 'systeme', 'resultat'])

# Filtrage : suppression stopwords + ponctuation
tokens_filtres = [t.lower() for t in tokens
                  if t.lower() not in sw_fr and t.isalpha()]

print(f"Avant ({len(tokens)}) : {tokens}")
print(f"Après ({len(tokens_filtres)}) : {tokens_filtres}")
# Avant (11) : ['Le', 'modele', 'analyse', 'les', 'donnees',
...]
```

```
# Apres ( 3 ) : ['analyse', 'multimodales', 'conference']
```

2.6 Normalisation Morphologique

2.6.1 Racinisation (Stemming)

Définition

La **racinisation** réduit chaque mot à son **radical** par des règles algorithmiques. Le résultat n'est pas toujours un mot valide de la langue.

Exemple : analyser, analysons, analyse → analys

Utilité

Rapide et sans lexique, la racinisation groupe les formes fléchies d'un même mot sous un radical unique, réduisant la dimensionnalité du vocabulaire. Elle convient aux grands corpus où la vitesse prime sur la précision linguistique.

Fonction	Fonctionnalité	Instruction
SnowballStemmer('fr')	Raciniseur multilingue (FR recommande)	SnowballStemmer('french')
stemmer.stem()	Obtenir le radical d'un mot	stemmer.stem('traitement')
PorterStemmer()	Raciniseur anglais classique	PorterStemmer()
LancasterStemmer()	Raciniseur agressif	LancasterStemmer()
SnowballStemmer.languages	Langues supportées	SnowballStemmer.languages

2.6.2 Lemmatisation

Définition

La **lemmatisation** retourne le **lemme**, c'est-à-dire la forme canonique du mot telle qu'elle figure dans le dictionnaire, en tenant compte de la catégorie grammaticale.

Exemple : running (verbe) → run ; better (adjectif) → good

Utilité

Contrairement à la racinisation, la lemmatisation produit toujours un mot valide. Elle est indispensable pour la traduction automatique, la recherche d'information et la génération de texte. Elle est plus lente car elle nécessite le lexique WordNet.

Fonction	Fonctionnalité	Instruction
WordNetLemmatizer()	Créer un lemmatiseur WordNet	WordNetLemmatizer()
lem.lemmatize(mot)	Lemmatiser (nom par défaut)	lem.lemmatize('running')
lem.lemmatize(mot, pos='v')	Lemmatiser un verbe	lem.lemmatize('running', pos='v')
lem.lemmatize(mot, pos='a')	Lemmatiser un adjectif	lem.lemmatize('better', pos='a')
SnowballStemmer('fr')	Alternative pour le français	stem.stem('traitement')

Listing 2.7: Pipeline complet nettoyage + stopwords + lemmatisation

```

import nltk, re
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer, SnowballStemmer

def pipeline_tal(texte: str, langue: str = 'french') -> list:
    """
    Pipeline TAL complet : nettoyage -> tokenisation ->
    stopwords -> lemmatisation.

    Args:
        texte : Texte brut
        langue : Langue du corpus ('french' ou 'english')

    Returns:
        Liste de tokens normalises
    """
    # 1. Nettoyage
    texte = texte.lower()
    texte = re.sub(r'[^\w\s]', ' ', texte)
    texte = re.sub(r'\d+', '', texte)
    texte = re.sub(r'\s+', ' ', texte).strip()

    # 2. Tokenisation
    tokens = word_tokenize(texte, language=langue)

    # 3. Filtrage stopwords
    sw = set(stopwords.words(langue))
    tokens = [t for t in tokens if t not in sw and t.isalpha()]

    # 4a. Lemmatisation (anglais via WordNet)
    if langue == 'english':
        lem = WordNetLemmatizer()
        tokens = [lem.lemmatize(t, pos='v') for t in tokens]
    # 4b. Stemming (français via Snowball)

```

```

else:
    stem = SnowballStemmer('french')
    tokens = [stem.stem(t) for t in tokens]

return tokens

# Exemple
resultat = pipeline_tal("Le traitement multimodal analyse les
    donnees visuelles.")
print(resultat)
# ['traitement', 'multimodal', 'analys', 'donnee', 'visuel']

```

2.7 Encodage et Embeddings

Définition

L'**encodage** convertit les tokens en vecteurs numériques exploitables par un modèle ML. Trois niveaux existent : la vectorisation classique (TF-IDF, BoW), les embeddings denses statiques (Word2Vec, GloVe) et les embeddings contextuels (BERT, CamemBERT).

Fonction	Fonctionnalité	Instruction
TfidfVectorizer()	Vectorisation TF-IDF	TfidfVectorizer(max_features=5000)
CountVectorizer()	Sac de mots	CountVectorizer(ngram_range=(1,2))
Word2Vec()	Embeddings Word2Vec	Word2Vec(sentences, vector_size=100)
KeyedVectors.load()	Charger GloVe pre-entraîne	KeyedVectors.load_word2vec_format()
AutoTokenizer	Tokeniseur BERT/-CamemBERT	AutoTokenizer.from_pretrained(...)
AutoModel	Modèle Transformers pre-entraîne	AutoModel.from_pretrained(...)

2.8 Analyse Exploratoire (EDA) du Texte

Fonction	Fonctionnalité	Instruction
FreqDist()	Distribution de fréquences	FreqDist(tokens)
freq.most_common(n)	N mots les plus fréquents	freq.most_common(20)
bigrams()	Bigrammes de tokens	list(bigrams(tokens))
ngrams(tokens, n)	N-grammes	list(ngrams(tokens, 3))

Fonction	Fonctionnalité	Instruction
WordCloud()	Nuage de mots (wordcloud)	WordCloud().generate(texte)
plt.bar()	Histogramme de fréquences	plt.bar(mots, freq)

Listing 2.8: Analyse exploratoire du corpus

```

from nltk import FreqDist, bigrams

# Frequences
freq = FreqDist(tokens_filtres)
print("Top 10 :", freq.most_common(10))

# Bigrammes les plus fréquents
bigr = FreqDist(bigrams(tokens_filtres))
print("Top bigrammes :", bigr.most_common(5))

```

2.9 Bonnes Pratiques

- **Toujours** appeler `nltk.download()` avant la première utilisation.
- Préciser `language='french'` dans `word_tokenize` et `sent_tokenize`.
- Appliquer `.lower()` **avant** la comparaison aux stopwords.
- Préférer `SnowballStemmer('french')` pour le français — `PorterStemmer` est conçu pour l'anglais.
- Vérifier l'encodage UTF-8 du fichier avant tout chargement.
- Traiter les valeurs manquantes et doublons **avant** la vectorisation.

Attention

Ne pas appliquer la lemmatisation WordNet sur du français : elle est basée sur le lexique anglais WordNet et produira des résultats incorrects. Pour le français, utiliser `SnowballStemmer('french')` ou un lemmatiseur dédié (`spaCy + fr_core_news_sm`).

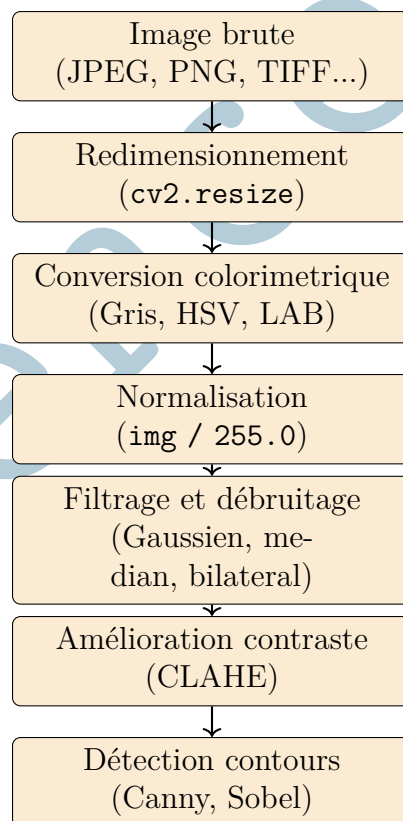
Chapitre 3 Prétraitement Visuel

3.1 Principe Fondamental

Le prétraitement visuel prépare les images brutes pour les modèles de vision par ordinateur. Photographies, scans médicaux, images satellitaires et frames vidéo doivent être standardisées en termes de dimensions, de plage de valeurs et de contraste avant tout apprentissage.

OpenCV (Open Source Computer Vision Library) est le standard industriel : plus de 2500 algorithmes optimisés, exposés en Python avec `numpy.ndarray` comme format natif. **Pillow** complète OpenCV pour la gestion des formats. **scikit-image** apporte des algorithmes supplémentaires pour l'analyse d'images scientifiques.

3.1.1 Architecture du Pipeline Vision



3.2 Installation

Listing 3.1: Installation des bibliothèques vision

```
pip install opencv-python pillow scikit-image
```



```
pip install opencv-contrib-python # Modules supplémentaires (
    SIFT, SURF)
```

Listing 3.2: Importation et verification

```
import cv2
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

print(cv2.__version__) # ex. 4.9.0
```

3.3 Redimensionnement et Transformations Géométriques

Définition

Le **redimensionnement** ajuste les dimensions d'une image a une taille cible. Les **transformations géométriques** (rotation, translation, retournement) modifient la géométrie de l'image sans changer le contenu. Ces opérations sont essentielles pour standardiser les entrées des réseaux de neurones convolutifs (CNN).

Utilité

Les modèles CNN exigent des images de taille fixe (224x224 pour ResNet, 299x299 pour Inception). Les transformations géométriques sont également utilisées pour l'**augmentation de données** : multiplier artificiellement le dataset par rotation, retournement et translation.

Fonction	Fonctionnalite	Instruction
cv2.resize()	Redimensionner a une taille cible	cv2.resize(img, (224,224))
cv2.resize() (facteur)	Redimensionner par facteur	cv2.resize(img, None, fx=0.5, fy=0.5)
cv2.getRotationMatr	Matrice de rotation 2D	cv2.getRotationMatrix2D(centre, 30, 1.0)
cv2.warpAffine()	Appliquer une transformation affine	cv2.warpAffine(img, M, (w,h))
cv2.flip()	Retournement horizontal/vertical	cv2.flip(img, 1)
cv2.INTER_CUBIC	Interpolation bicubique (qualité)	interpolation=cv2.INTER_CUBIC
cv2.INTER_AREA	Interpolation pour réduction	interpolation=cv2.INTER_AREA

Listing 3.3: Redimensionnement et transformations geometriques

```
import cv2
```

```

import numpy as np

img = cv2.imread('scene.jpg')
h, w = img.shape[:2]

# 1. Resize cible CNN (224x224)
img_224 = cv2.resize(img, (224, 224),
                      interpolation=cv2.INTER_CUBIC)

# 2. Resize par facteur (50%)
img_demi = cv2.resize(img, None, fx=0.5, fy=0.5,
                      interpolation=cv2.INTER_AREA)

# 3. Rotation de 30 degres autour du centre
centre = (w // 2, h // 2)
M_rot = cv2.getRotationMatrix2D(centre, angle=30, scale=1.0)
img_rot = cv2.warpAffine(img, M_rot, (w, h))

# 4. Retournement horizontal (miroir)
img_flip = cv2.flip(img, flipCode=1)

# 5. Translation (+50, +30) pixels
M_trans = np.float32([[1, 0, 50], [0, 1, 30]])
img_trans = cv2.warpAffine(img, M_trans, (w, h))

print("Forme finale :", img_224.shape)    # (224, 224, 3)

```

3.4 Conversion Colorimétrique et Normalisation

Définition

La **conversion colorimétrique** transforme une image d'un espace de couleur à un autre. La **normalisation** ramène les valeurs des pixels dans la plage $[0, 1]$ ou applique une standardisation z-score, afin d'uniformiser la magnitude pour les modèles ML.

Fonction	Fonctionnalité	Instruction
cv2.cvtColor (BGR2GRAY)	Conversion en niveaux de gris	cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.cvtColor (BGR2RGB)	Conversion BGR vers RGB	cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
cv2.cvtColor (BGR2HSV)	Conversion vers HSV	cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
cv2.cvtColor (BGR2LAB)	Conversion vers LAB	cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
astype(float32) / 255.0	Normalisation $[0, 1]$	img.astype(np.float32) / 255.0

Fonction	Fonctionnalité	Instruction
<code>cv2.threshold()</code>	Seuillage binaire	<code>cv2.threshold(gray, 127, 255, THRESH_BINARY)</code>
<code>cv2.adaptiveThreshold()</code>	Seuillage adaptatif local	<code>cv2.adaptiveThreshold(gray, 255, ...)</code>

Remarque

OpenCV charge les images en **BGR** (Blue-Green-Red) et non RGB. Avant tout affichage avec Matplotlib (`plt.imshow()`), convertir obligatoirement avec `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

3.5 Filtrage et Réduction de Bruit

Définition

Le **filtrage spatial** atténue le bruit et lisse les textures par convolution d'un noyau sur l'image. Chaque pixel de sortie est calculé à partir de ses voisins, avec des poids définis par le noyau.

Utilité

Le filtrage réduit le bruit gaussien (`GaussianBlur`), le bruit sel-et-poivre (`medianBlur`) et préserve les contours importants (`bilateralFilter`). Il est indispensable avant la détection de contours et les opérations de morphologie mathématique.

Fonction	Fonctionnalité	Instruction
<code>cv2.GaussianBlur()</code>	Lissage gaussien (bruit général)	<code>cv2.GaussianBlur(img, (5,5), 0)</code>
<code>cv2.medianBlur()</code>	Filtre médian (bruit sel-poivre)	<code>cv2.medianBlur(img, 5)</code>
<code>cv2.bilateralFilter</code>	Lissage préservant les contours	<code>cv2.bilateralFilter(img, 9, 75, 75)</code>
<code>cv2.filter2D()</code>	Convolution avec noyau libre	<code>cv2.filter2D(img, -1, kernel)</code>
<code>cv2.Laplacian()</code>	Détection contours (dérivée 2e)	<code>cv2.Laplacian(gray, cv2.CV_64F)</code>

Attention

La taille du noyau doit toujours être un entier **impair** (3, 5, 7, 9...). Une taille paire produit une erreur `cv2.error`. Plus le noyau est grand, plus le lissage est fort mais plus les contours sont dégradés.

3.6 Amélioration du Contraste

Définition

L'égalisation d'histogramme redistribue les niveaux d'intensité pour maximiser le contraste. Le **CLAHE** (Contrast Limited Adaptive Histogram Equalization) applique cette égalisation localement sur des tuiles, en limitant le gradient pour éviter le sur-contraste global.

Utilité

Le CLAHE est indispensable pour les images médicales (IRM, radiographies), les photos en faible lumière et les frames vidéo sombres. Il amélioré significativement la qualité des features extraites par les modèles CNN.

Fonction	Fonctionnalité	Instruction
<code>cv2.equalizeHist()</code>	Égalisation globale d'histogramme	<code>cv2.equalizeHist(gray)</code>
<code>cv2.createCLAHE()</code>	Créer un objet CLAHE	<code>cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))</code>
<code>clahe.apply()</code>	Appliquer le CLAHE	<code>clahe.apply(gray)</code>
<code>cv2.calcHist()</code>	Calculer l'histogramme	<code>cv2.calcHist([gray], [0], None, [256], [0, 256])</code>

Listing 3.4: Pipeline complet de pretraitement visuel

```
import cv2
import numpy as np

def pretraiter_image(chemin: str) -> np.ndarray:
    """
    Pipeline complet de pretraitement pour une image destinee
    a un CNN.

    Etapes :
    1. Lecture et verification
    2. Redimensionnement 224x224
    3. Conversion en niveaux de gris
    4. Filtrage gaussien (debruitage)
    5. Egalisation CLAHE (contraste)
    6. Normalisation [0, 1]

    Returns:
        Tableau numpy float32 de forme (224, 224)
    """
    # 1. Lecture
    img = cv2.imread(chemin)
    if img is None:
```

```

    raise FileNotFoundError(f"Image introuvable :
                           {chemin}")

# 2. Redimensionnement
img = cv2.resize(img, (224, 224),
                 interpolation=cv2.INTER_CUBIC)

# 3. Conversion en niveaux de gris
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 4. Filtrage gaussien
gray = cv2.GaussianBlur(gray, (5, 5), 0)

# 5. CLAHE
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
gray = clahe.apply(gray)

# 6. Normalisation [0, 1]
gray_norm = gray.astype(np.float32) / 255.0

print(f"Forme      : {gray_norm.shape}")           # (224, 224)
print(f"Min/Max   : {gray_norm.min():.3f} /
      {gray_norm.max():.3f}")

return gray_norm

# Exemple
img_ok = pretraiter_image("portrait.jpg")

```

3.7 Détection de Contours

Définition

La **détection de contours** identifie les frontières entre régions dans une image en détectant les variations brusques d'intensité. Elle extrait les structures visuelles porteuses d'information (bords d'objets, formes, textes).

Fonction	Fonctionnalité	Instruction
cv2.Canny()	Détection Canny (recommandée)	cv2.Canny(gray, 100, 200)
cv2.Sobel()	Gradient directionnel (Sobel)	cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
cv2.findContours()	Extraire les contours fermes	cv2.findContours(bin, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE)
cv2.drawContours()	Dessiner les contours sur l'image	cv2.drawContours(img, contours, -1, (0,255,0), 2)

3.8 Bonnes Pratiques

- Toujours convertir BGR→RGB avant tout affichage Matplotlib.
- Normaliser `/255.0` en float32 avant tout modèle CNN.
- Préférer CLAHE à `equalizeHist` pour éviter le sur-contraste.
- La taille du noyau de filtre doit toujours être un entier impair.
- Sauvegarder les images de référence en PNG (sans perte) plutôt qu'en JPEG.
- Vérifier que `cv2.imread` ne retourne pas `None` avant tout traitement.

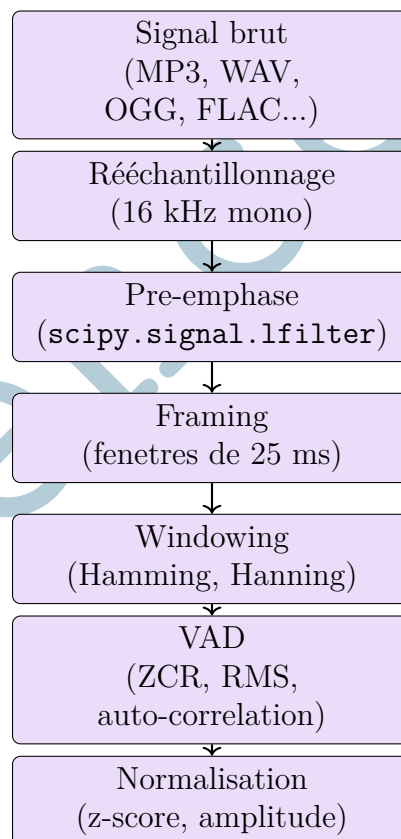
Chapitre 4 Prétraitement Audio

4.1 Principe Fondamental

Le prétraitement audio transforme des signaux bruts et hétérogènes en représentations normalisées exploitables par les modèles d'apprentissage. Enregistrements vocaux, podcasts, signaux physiologiques et bandes-son de vidéos sont traités par une chaîne dédiée.

Pydub gère les opérations haut niveau (chargement multi-format via FFmpeg, découpage, normalisation de volume, export). **Librosa** est la référence pour l'analyse spectrale (MFCC, spectrogramme mel, ZCR, RMS). **SciPy** (`scipy.signal`) assure le filtrage numérique précis (Butterworth, pre-emphase).

4.1.1 Architecture du Pipeline Audio



4.2 Installation

Listing 4.1: Installation des bibliothèques audio

```
pip install pydub librosa scipy pyaudio
# FFmpeg requis pour MP3, M4A, OGG, FLAC
```

```
# Ubuntu/Debian : sudo apt install ffmpeg
# Windows      : telecharger depuis ffmpeg.org et ajouter au
PATH
```

Listing 4.2: Importation des modules

```
from pydub import AudioSegment
from pydub.silence import split_on_silence, detect_silence
import librosa
import numpy as np
from scipy import signal
from sklearn.preprocessing import StandardScaler
```

Attention

Pydub nécessite **FFmpeg** installé sur le système pour charger les formats MP3, M4A, OGG et FLAC. Sans FFmpeg, seul le format WAV est supporté nativement. Une erreur `CouldntDecodeError` indique que FFmpeg est absent ou mal configuré.

4.3 Rééchantillonnage

Définition

Le **rééchantillonnage** modifie la fréquence d'échantillonnage d'un signal audio. D'après le **théorème de Nyquist-Shannon**, la fréquence d'échantillonnage doit être au moins le double de la fréquence maximale du signal : $f_e \geq 2 \times f_{max}$.

Utilité

La majorité des modèles ASR (Whisper, Wav2Vec, DeepSpeech) exigent un signal mono à 16 kHz. La standardisation de f_e est la **première étape obligatoire** de tout pipeline audio destiné à la reconnaissance vocale ou à l'extraction de features MFCC.

Fonction	Fonctionnalité	Instruction
<code>set_frame_rate()</code>	Rééchantillonner avec Pydub	<code>audio.set_frame_rate(16000)</code>
<code>set_channels()</code>	Convertir en mono	<code>audio.set_channels(1)</code>
<code>librosa.resample()</code>	Rééchantillonner avec Librosa	<code>librosa.resample(y, orig_sr=44100, target_sr=16000)</code>
<code>resample_poly()</code>	Rééchantillonner avec SciPy	<code>resample_poly(samples, 160, 441)</code>
<code>audio.frame_rate</code>	Lire la fréquence courante	<code>audio.frame_rate</code>
<code>audio.channels</code>	Lire le nombre de canaux	<code>audio.channels</code>

Listing 4.3: Standardisation audio pour ASR

```

from pydub import AudioSegment

def standardiser_audio(chemin: str,
                      freq_cible: int = 16000) ->
                      AudioSegment:
    """
    Standardise un fichier audio pour les pipelines ASR.

    Etapes :
    1. Chargement multi-format (via FFmpeg)
    2. Conversion en mono
    3. Reechantillonnage a la frequence cible

    Args:
        chemin      : Chemin du fichier audio (MP3, WAV, OGG,
                     FLAC)
        freq_cible  : Frequence d'echantillonnage cible (16000
                     Hz par default)

    Returns:
        AudioSegment standardise
    """
    audio = AudioSegment.from_file(chemin)

    print(f"Format initial : {audio.frame_rate} Hz,
          {audio.channels} canaux")

    # Conversion mono
    audio = audio.set_channels(1)

    # Reechantillonnage
    audio = audio.set_frame_rate(freq_cible)

    print(f"Apres          : {audio.frame_rate} Hz,
          {audio.channels} canal")
    print(f"Duree          : {audio.duration_seconds:.2f} s")

    return audio

# Exemple
audio_ok = standardiser_audio("interview.mp3")
audio_ok.export("interview_16k.wav", format="wav")

```

4.4 Pre-emphase

Définition

La **pre-emphase** est un filtrage passe-haut applique au signal de parole pour am-

plifier les hautes fréquences. Elle est modélisée par le filtre de premier ordre : $y[n] = x[n] - \alpha \cdot x[n-1]$ avec $\alpha \approx 0.97$.

Utilité

La parole humaine a naturellement plus d'énergie dans les basses fréquences. La pré-emphase équilibre le spectre, améliorant le rapport signal-a-bruit pour les hautes fréquences et la précision des features spectrales (MFCC, LPC).

Fonction	Fonctionnalité	Instruction
<code>signal.lfilter()</code>	Filtre IIR de pré-emphase	<code>signal.lfilter([1, -0.97], 1, samples)</code>
<code>signal.butter()</code>	Concevoir un filtre Butterworth	<code>signal.butter(N=4, Wn=fc/(fs/2), btype='low')</code>
<code>signal.filtfilt()</code>	Filtrage sans déphasage	<code>signal.filtfilt(b, a, samples)</code>
<code>signal.freqz()</code>	Réponse en fréquence du filtre	<code>signal.freqz(b, a)</code>

Listing 4.4: Pre-emphase et filtrage

```
import numpy as np
from scipy import signal
from pydub import AudioSegment

audio = AudioSegment.from_file('parole_16k.wav')
samples = np.array(audio.get_array_of_samples(),
                    dtype=np.float64)
fs = audio.frame_rate # 16000 Hz

# Pre-emphase (filtre passe-haut premier ordre, alpha=0.97)
samples_emp = signal.lfilter([1, -0.97], 1, samples)

# Filtre passe-bas (coupure a 4000 Hz -- bande de la parole)
b, a = signal.butter(N=4, Wn=4000/(fs/2), btype='low')
samples_flt = signal.filtfilt(b, a, samples_emp)

# Normalisation amplitude [-1, 1]
samples_norm = samples_flt / np.max(np.abs(samples_flt))

print(f"Echantillons : {len(samples_norm)}")
print(f"Duree : {len(samples_norm)/fs:.2f} s")
```

4.5 Framing et Windowing

Définition

Le **framing** découpe le signal audio en segments courts (20–30 ms) sur lesquels on suppose la **stationnarité locale** du signal. Le **windowing** applique une fonction de fenêtre sur chaque frame pour atténuer les discontinuités aux bords avant la transformée de Fourier (FFT).

Utilité

Sans framing, le signal audio est non-stationnaire sur l'ensemble de sa durée. Le framing permet de calculer des features spectrales (MFCC, FFT) sur des segments où le signal est approximativement stationnaire. Le windowing (Hamming) réduit le biais spectral lors de la FFT.

Fonction	Fonctionnalité	Instruction
<code>np.hamming(L)</code>	Fenêtre de Hamming (ASR standard)	<code>fenetre = np.hamming(400)</code>
<code>np.hanning(L)</code>	Fenêtre de Hanning (musique)	<code>fenetre = np.hanning(400)</code>
<code>np.blackman(L)</code>	Fenêtre de Blackman (lobes réduits)	<code>fenetre = np.blackman(400)</code>
<code>np.fft.rfft()</code>	FFT réelle sur une frame	<code>np.fft.rfft(frame * fenetre)</code>
<code>librosa.stft()</code>	STFT (Short-Time Fourier Transform)	<code>librosa.stft(y, n_fft=512)</code>

Listing 4.5: Framing et windowing avec numpy

```
import numpy as np
from pydub import AudioSegment

# Chargement et standardisation
audio = (AudioSegment.from_file('parole.wav')
         .set_channels(1).set_frame_rate(16_000))
samples = np.array(audio.get_array_of_samples(),
                   dtype=np.float64)
fs = 16_000

# Parametres de framing (standard ASR)
taille_frame = int(0.025 * fs) # 25 ms = 400 echantillons
pas = int(0.010 * fs) # 10 ms = 160 echantillons
    (overlap 60%)

# Decoupage en frames
frames = [samples[i : i + taille_frame]
          for i in range(0, len(samples) - taille_frame, pas)]

# Application de la fenetre de Hamming
```

```
fenetre          = np.hamming(taille_frame)
frames_fenetres = np.array([f * fenetre for f in frames])

print(f"Nombre de frames : {len(frames_fenetres)}")
print(f"Forme du tableau : {frames_fenetres.shape}")
# Forme : (N_frames, 400) <- pret pour FFT / MFCC
```

4.6 Détection d'activité Vocale (VAD)

Définition

La **VAD** (Voice Activity Détection) distingue les segments de parole active des segments de silence ou de bruit de fond. Elle repose sur trois critères : le taux de passage par zero (ZCR), l'énergie a court terme (RMS) et l'auto-corrélation.

Fonction	Fonctionnalité	Instruction
zero_crossing_rate()	Taux de passage par zéro	librosa.feature.zero_crossing_rate(y)
rms()	Énergie RMS par frame	librosa.feature.rms(y=y)
np.correlate()	Auto-corrélation	np.correlate(frame, frame, mode='full')
split_on_silence()	Segmenter sur les silences	split_on_silence(audio, min_silence_len=500)
detect_silence()	Détecter les plages vides	detect_silence(audio, silence_thresh=-40)

4.7 Normalisation du Signal

Définition

La **normalisation audio** réduit la variabilité inter-locuteurs et inter-conditions d'enregistrement. La normalisation z-score (zero mean, unit variance) est la méthode la plus utilisée pour la préparation des features MFCC.

Fonction	Fonctionnalité	Instruction
StandardScaler()	Normalisation z-score (mean=0, std=1)	StandardScaler().fit_transform(features)
audio.normalize()	Normalisation de volume Pydub	audio.normalize()
/np.max(np.abs(samp	Normalisation amplitude [-1, 1]	s = s / np.max(np.abs(s))

Fonction	Fonctionnalite	Instruction
audio + gain	Ajustement gain en dB (Pydub)	audio + 6
audio.dBFS	Niveau dB full-scale courant	audio.dBFS

Listing 4.6: Pipeline audio complet avec extraction MFCC

```

import numpy as np
import librosa
from sklearn.preprocessing import StandardScaler
from pydub import AudioSegment
from scipy import signal

def pipeline_audio(chemin: str) -> np.ndarray:
    """
    Pipeline audio complet : standardisation -> pre-emphase
    -> framing -> windowing -> MFCC -> normalisation.

    Returns:
        Matrice de features MFCC normalisees, forme (T, 13)
    """
    # 1. Chargement et standardisation
    audio = (AudioSegment.from_file(chemin)
              .set_channels(1).set_frame_rate(16_000))
    samples = np.array(audio.get_array_of_samples(),
                       dtype=np.float64)
    samples = samples / np.max(np.abs(samples)) #
        normalisation amplitude
    fs = 16_000

    # 2. Pre-emphase
    samples = signal.lfilter([1, -0.97], 1, samples)

    # 3. Extraction MFCC avec Librosa
    mfcc = librosa.feature.mfcc(y=samples.astype(np.float32),
                                sr=fs,
                                n_mfcc=13,
                                n_fft=512,
                                hop_length=160,
                                win_length=400,
                                window='hamming')

    # mfcc : (13, T) --> transposer en (T, 13)
    mfcc = mfcc.T

    # 4. Normalisation z-score (mean=0, std=1)
    mfcc_norm = StandardScaler().fit_transform(mfcc)

    print(f"Forme MFCC normalises : {mfcc_norm.shape}")

```

```
return mfcc_norm

# Exemple
features = pipeline_audio("discours.wav")
```

4.8 Bonnes Pratiques

- Toujours convertir en **mono 16 kHz** avant tout pipeline ASR ou extraction MFCC.
- Appliquer la pre-emphase **avant** le framing, sur le signal entier.
- Toujours fenêtrer (Hamming) les frames **avant** la FFT pour réduire le biais spectral.
- Vérifier `audio.channels` et `audio.frame_rate` après chaque operation Pydub.
- Sauvegarder les parametres (taille_frame, pas, alpha) dans un fichier JSON pour la reproductibilité.
- Utiliser `StandardScaler` plutôt qu'une normalisation min-max pour les features MFCC (la z-norm est plus robuste aux outliers).

Attention

Le signal retourne par `AudioSegment.get_array_of_samples()` est en entiers (int16 ou int32). Il faut le convertir en float64 avec `samples.astype(np.float64)` avant tout calcul numérique. Diviser par l'amplitude maximale (`/ np.max(np.abs(samples))`) pour travailler dans $[-1, 1]$.

Chapitre 5 Synthèse et Bonnes Pratiques

5.1 Tableau Comparatif des Techniques de Prétraitement

Modalite	Technique	Objectif	Outil Python	Sortie
Texte	Nettoyage	Supprimer artefacts	<code>re, str.lower()</code>	str nettoyée
	Tokenisation	Decouper en tokens	<code>word_tokenize</code>	list[str]
	Normalisation	Forme canonique	<code>SnowballStemmer</code>	tokens normalisés
	Encodage	Vecteur numérique	<code>TfidfVectorizer</code>	matrice sparse
Image	Redimensionnement	Standardiser dimensions	<code>cv2.resize</code>	ndarray (H,W,C)
	Normalisation	Plage [0, 1]	<code>img / 255.0</code>	float32
	CLAHE	Amélioration contraste	<code>cv2.createCLAHE</code>	ndarray amélioré
	Contours	Extraction structures	<code>cv2.Canny</code>	ndarray binaire
Audio	Rééchantillonnage	Standardiser f_e	<code>pydub / librosa</code>	AudioSegment
	Pre-emphase	Accentuer hautes freq.	<code>scipy.signal.lf</code>	array numpy
	Framing + Window	Stationnarité locale	<code>numpy</code>	matrice (N, L)
	Normalisation z	Reduire variabilité	<code>StandardScaler</code>	features norm.

5.2 Erreurs Frequentes et Solutions

Outil	Erreur fréquente	Symptome	Correction
NLTK	LookupError: punkt	Crash à l'exécution	<code>nltk.download('punkt')</code> avant appel

Outil	Erreur fréquente	Symptôme	Correction
NLTK	Stopwords non filtres	Mots résiduels	Appliquer <code>.lower()</code> avant comparaison
NLTK	Lemmatisation FR incorrecte	Lemmes anglais	Utiliser <code>SnowballStemmer('french')</code>
sklearn	ValueError TF-IDF	Corpus vide	Vérifier <code>dropna()</code> avant vectorisation
OpenCV	Couleurs bleutees	Affichage incorrect	Convertir BGR→RGB avant <code>imshow</code>
OpenCV	<code>imread</code> retourne None	Crash sur <code>.shape</code>	Vérifier chemin, existence et droits fichier
OpenCV	Erreur cv2 sur kernel	Kernel de taille paire	Utiliser uniquement des tailles impaires (3,5,7...)
Pydub	CouldntDecodeError	Chargement MP3 echoue	Installer FFmpeg sur le système (apt/choco)
scipy	Signal quasi-nul	Attenuation excessive	Reduire ordre N, vérifier f_c normalisee
Librosa	TypeError float64	Types incompatibles	Convertir avec <code>samples.astype(np.float32)</code>

5.3 Checklist de Reproductibilite

- ☐ Fixer la graine aléatoire : `np.random.seed(42)`.
- ☐ Sauvegarder tous les paramétrés du pipeline dans un fichier JSON.
- ☐ Versionner `requirements.txt` avec les versions exactes des bibliothèques.
- ☐ Tester chaque étape du pipeline isolément avant l'intégration.
- ☐ Relancer « Kernel + Run All » avant toute remise de notebook Jupyter.
- ☐ Documenter la provenance de chaque dataset et les transformations appliquées.
- ☐ Vérifier que le pipeline produit le même résultat sur des exécutions successives.

5.4 Ressources Complémentaires

- **NLTK Book Online** (nltk.org/book) — Référence complète sur le TAL avec NLTK, disponible gratuitement en ligne.
- **OpenCV Documentation** (docs.opencv.org) — Documentation officielle avec tutoriels et exemples pour la vision par ordinateur.
- **Librosa Documentation** (librosa.org/doc) — Référence pour l'analyse musicale et audio en Python.
- **HuggingFace Hub** (huggingface.co/models) — Accès à des milliers de modèles pré-entraînés (BERT, Whisper, CamemBERT) pour le texte et l'audio.
- **Kaggle** (kaggle.com/datasets) — Datasets multimodaux pour pratiquer les

pipelines de prétraitement.

Conseil

Pour un démarrage rapide :

- Texte (français) : NLTK avec `punkt` + `stopwords` + `SnowballStemmer`
- Images : `opencv-python-headless` (compatible serveurs et Google Colab)
- Audio : `pydub` + `ffmpeg` system (`sudo apt install ffmpeg`)